M2 - Architecture et Programmation d'accélérateurs Matériels.

(APM 2026-2027)

TP3

GPU Occupancy - Block Tiling

adrien.roussel@cea.fr

Les objectifs de ce TP sont :
— Savoir calculer l'occupation d'un kernel sur le GPU,
— Optimiser un kernel pour accroître l'occupation sur GPU.

I Multiplication de matrices (DGEMM)

Lors du TP précédent, nous avons vu comment écrire notre premier kernel effectuant la multiplication entre deux matrice de manière optimisée en favorisant les accès coalescés et en utilisant de la mémoire partagée. Nous nous posons la question suivante : est-ce que mon kernel est suffisamment optimisé ou est-ce que je peux aller encore plus loin ? Et si oui, comment le détecter et optimiser ? Nous allons maintenant analyser et comprendre ce code afin de pouvoir répondre à cette question.

1 GPU Occupancy

L'occupation (occupancy) est définie comme le ratio entre le nombre de warps actifs par SM et le nombre maximum possible de warps actifs par SM. Une occupation élevée est utile car elle nous permet de masquer la forte latence de nos opérations, en disposant d'un ensemble plus large d'instructions prêtes à être émises. Il existe trois limites principales au maintien d'un plus grand nombre de blocs actifs chargés sur un SM : le nombre de registres, le nombre de warps et la capacité en mémoire partagée (SMEM).

1.1 La limite de la mémoire partagée (SMEM)

Pour savoir si la quantité de mémoire partagée est un facteur limitant à l'occupation sur GPU, il nous faut calculer le ratio suivant :

$$\text{Blocs}_{\text{SMEM}} = \left\lfloor \frac{\text{Taille SMEM disponible par SM}}{\text{SMEM demandée par bloc}} \right\rfloor$$

Ces informations peuvent être connues au runtime, et ainsi être calculées.

Q.1: Pour connaitre les informations relatives au GPU utilisées, l'API CUDA propose de l'interroger directement via la fonction `cudaGetDeviceProperties`. Grâce à cette dernière, obtenez la taille maximale de SMEM disponible par SM ainsi que la taille maximale de SMEM allouable par un seul bloc.

Q.2: Maintenant, il nous faut la quantité de mémoire partagée demandée par bloc relative au kernel que l'on exécute. La fonction `cudaFuncGetAttributes` nous fournit cette donnée. Affichez la.

Q.3: A partir des deux questions précédentes, calculez la quantité maximale physique de blocs que la mémoire partagée du SM peut contenir simultanément avant de saturer (i.e., le ratio $\text{Blocs}_{\text{SMEM}}$ défini plus haut).

1.2 Threads

Pour savoir si la quantité de threads demandée est un facteur limitant à l'occupation sur GPU, il nous faut calculer le ratio suivant :

$$\text{Blocs}_{\text{SMEM}} = \left\lfloor \frac{\text{Nb max de Threads par SM}}{\text{Nb threads demandés par bloc}} \right\rfloor$$

Q.4: Toujours grâce à la fonction `cudaGetDeviceProperties`, récupérer le nombre maximal de threads par SM.

Q.5: Maintenant, calculez le nombre maximum théorique de blocs qu'un SM peut exécuter simultanément à partir du nombre de threads par blocs que vous utilisez

1.3 La limite des registres

Pour savoir si la quantité de registres alloués est un facteur limitant à l'occupation sur GPU, il nous faut calculer le ratio suivant :

$$\text{Blocs}_{\text{SMEM}} = \left\lfloor \frac{\text{Nb de registres par SM}}{\text{Nb registres alloués par bloc}} \right\rfloor$$

Q.6: Pour savoir le nombre de registres que l'on utilise pour exécuter un kernel, la fonction `cudaFuncGetAttributes` peut nous y aider. Affichez cette valeur par thread, et calculez la valeur par warp.

Q.7: La Register Allocation Unit Size (ou taille de l'unité d'allocation des registres) est la granularité minimale avec laquelle le matériel (le GPU) alloue la mémoire de ses registres à un ensemble de threads (un warp). Bien que NVIDIA ait fixée sa valeur à 256, elle dépend tout de même de la génération du GPU sur lequel nous exécutons notre programme. La fonction ci-dessous permet de récupérer le nombre total de registres alloués par warp. A partir de cela, définissez le nombre de warps qu'il y a par bloc, et donnez ensuite le nombre de registres par blocs

```
1  // Renvoie la taille de l'éunit d'allocation des registres par Warp
2  int getRegisterAllocationUnitSize(int major, int minor) {
3      // Pour toutes les architectures modernes
4      // (Kepler 3.5 jusquà' Hopper/Ada 9.0+)
5      // L'allocation des registres se fait par
6      // blocs de 256 registres par warp.
7      if (major >= 3) {
8          return 256;
9      }
10     // Architectures ètrs anciennes (Fermi 2.x)
11     return 64;
```

```
12 }
13
14 size_t roundup_x(size_t val, int val2) {
15     return ((val + (val2-1)) / val2) * val2;
16 }
17
18 ...
19
20 void main()
21 {
22   ...
23   int reg_unit_size =
24     getRegisterAllocationUnitSize(prop.major, prop.minor);
25   int real_reg_per_warp =
26     roundup_x(attr.numRegs * prop.warpSize, reg_unit_size);
27   ...
28 }
```

Q.8: `cudaGetDeviceProperties` peut nous donner le nombre maximal de registres allouables par SM, nous venons de récupérer le nombre de registres nécessaires par blocs. En considérant cette contrainte, combien est-ce que l'on peut exécuter de blocs en même temps sur notre programme ?

1.4 Occupation sur GPU

Nous avons maintenant nos trois plafonds correspondant à un nombre de blocs maximum que l'on peut exécuter sous trois contraintes :

— la quantité de mémoire partagée utilisée,
— le nombre de threads,
— la quantité de registres utilisée.

Á partir de ces valeurs, le minimum entre les trois nous donne la contrainte la plus forte et qui limite notre occupation sur le GPU.

L'occupation du GPU se calcule alors via la formule suivante :

$$\text{Occupancy} = \frac{\text{Warps Actifs par SM}}{\text{Warps Max par SM}} \times 100$$

Q.9: Calculez le nombre maximal de warps actifs par SM à partir des propriétés du device.

Q.10: A partir du minimum entre les trois nombres de blocs calculés précédemment, calculez combien de warps peuvent être actifs en même temps sur un SM.

Q.11: Calculez l'occupancy.

1.5 Vérification par l'API CUDA

Q.12: Via l'API CUDA, l'occupation du GPU peut être directement connue via la fonction suivante :
`cudaOccupancyMaxActiveBlocksPerMultiprocessor`
Vérifiez sa valeur par rapport à celle que vous avez calculer à la question précédente.

1.6 Conclusion

Q.13: Que concluez-vous de l'occupation du kernel sur GPU ?

2 Analyse statique

Nous allons maintenant jeter un œil l'assembleur du kernel pour voir quelles opérations sont les plus exécutées. Pour cela, il nous faut extraire dans un premier temps le *SASS* (i.e., Source code Assembly). A partir de l'exécutable, nous pouvons l'extraire via cette commande :

```
cuobjdump -sass mon_programme > mon_kernel.sass
```

Q.14: Essayez d'extraire le SASS de la dernière version de DGEMM que nous avons implémentée.

Q.15: A partir du script `parse_sass.sh`, donnez le type d'opération qui est le plus exécuté et à quoi correspond chaque type. Ce script génére un tableau au format Markdown, qui trie de l'opération la plus exécutée à la moins exécutée.

Q.16: Une instruction type LDS coûte entre 20 et 40 cycles, et une FMA entre 4 et 8 cycles. Quelle conclusion pouvez-vous faire ?

II Optimisation

Q.17: A partir du constat que vous avez effectué lors de la question précédente, augmentez l'intensité arithmétique par thread pour améliorer les performances.